QA 자동화는 요즘 어떻게 하나 & QA_Automation_KC는 어떻게 동작하나

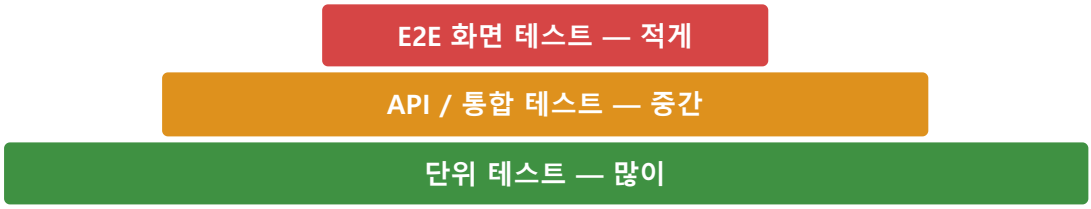
1부: 2026년 QA 자동화 개요 · 2부: QA_Automation_KC 코드 구조와 동작 · 3부: 비교와 다음 할 일
기준: v0.013 + 현재 작업 중인 코드(커밋 전 변경 포함) · 2026-11-10

> **한 줄 요약.** QA 자동화는 코드가 바뀔 때마다 자동으로 검사를 돌려, 사용자보다 먼저 문제를 찾는 것입니다. QA_Automation_KC는 그중 **E2E(화면) 테스트를 사람 대신 만들어 주는 프로그램**입니다. 사이트를 직접 눌러 보며 관찰한 결과를 Playwright 테스트 코드로 만들고, 그 코드를 여러 브라우저에서 다시 실행해 비교합니다.

1부

요즘 QA 자동화는 이렇게 합니다

1. 무엇을 테스트하나: 테스트 피라미드



계층	확인하는 것	속도	개수
단위	함수 · 클래스 하나	밀리초	수천 개
API / 통합	서비스 · DB · API가 함께 동작하는지	초	수백 개
E2E	실제 브라우저에서 사용자처럼 동작	느림	핵심 흐름만

검사는 빠른 아래 계층에 최대한 많이 두고, 로그인 · 가입 · 검색 · 결제 같은 핵심 흐름만 E2E로 확인합니다.

2. 많이 쓰는 도구

분야	도구	비고
웹 화면	**Playwright**, Cypress, Selenium	새 프로젝트는 대부분 Playwright. 3개 브라우저 엔진, 자동 대기, 트레이스
모바일 앱	Appium, Maestro, Espresso, XCUITest	실기기 또는 디바이스 클라우드
API	Playwright API, REST Assured, pytest, Postman	빠르고 안정적. 테스트 데이터 준비에도 사용
성능	k6, JMeter, Gatling, Core Web Vitals	부하 테스트, LCP 등 체감 속도
접근성	axe-core	WCAG 기준 자동 점검
화면 비교	Playwright 스냅샷, Percy, Applitools	승인된 기준 스크린샷과 비교

3. 어떻게 돌리나: CI/CD

```
코드 푸시 → 파이프라인 시작 (GitHub Actions · GitLab CI · Jenkins)
  1. 정적 검사 + 단위 테스트            (약 1분)
  2. 빌드 → 임시 테스트 환경 배포        (Docker · 프리뷰 배포)
  3. API / 통합 테스트
  4. E2E 스모크 테스트 (브라우저별 병렬 실행)
  5. 리포트 게시 (HTML · 스크린샷 · 영상 · 트레이스)
  6. 하나라도 실패하면 병합(merge) 차단
```

- **PR마다:** 단위 + API + 짧은 E2E 스모크
- **매일 밤:** 전체 회귀 테스트(여러 브라우저 · 기기)
- **배포 전:** 성능 · 보안 점검

4. 요즘 지키는 원칙

- **Shift-left:** 기능을 만들 때 테스트도 같이 만듭니다.
- **안정적인 로케이터:** CSS · XPath 대신 역할(role) · 라벨 · `data-testid`로 찾습니다.
- **상태 격리:** 테스트마다 깨끗한 상태로 시작하고, 쓴 데이터는 직접 정리합니다.
- **고정 대기 금지:** `sleep` 대신 자동 대기와 명시적인 확인을 씁니다.
- **불안정(flaky) 테스트 관리:** 기록하고 격리해서 고칩니다. 무시하지 않습니다.
- **비밀값 보호:** 계정과 키는 CI 시크릿 · 볼트에 둡니다. 로그인은 테스트 계정 + 저장된 세션(storageState)으로 합니다.

5. 가장 큰 변화: AI

기능	내용
AI가 테스트 작성	Claude Code 같은 도구가 앱이나 기획서를 읽고 테스트를 쓴 뒤, 직접 실행하고 고칩니다.
브라우저 에이전트	AI가 실제 브라우저를 조작해 탐색하고, 버그를 재현하고, 수동 TC를 자동화합니다 (Playwright MCP 등).
자가 치유 로케이터	UI가 바뀌어도 요소를 다시 찾아 테스트가 깨지지 않게 합니다.
실패 분류	로그 · 스크린샷 · 트레이스를 보고 실제 버그, 불안정, 환경 문제 중 무엇인지 구분합니다.

> QA 엔지니어의 일은 스크립트를 손으로 쓰는 쪽에서 **테스트 전략 · 위험 분석 · AI가 만든 테스트 검토** 쪽으로 옮겨 가고 있습니다.

2부

QA_Automation_KC는 이렇게 동작합니다

1. 전체 흐름

① 입력
URL · 기획서 · 범위 · 브라우저 · 로그인
`public/index.html`

② 페이지 찾기
같은 사이트 링크를 따라감 (최대 50페이지)
`generate-specs.js`

③ 화면 분석
요소를 찾아 실제로 눌러 보고 결과 관찰
`generate-specs.js`

④ 테스트 생성
관찰 결과 → *.spec.ts + cases.html
`lib/codegen.js`

⑤ 실행
새 브라우저에서 다시 해 보고 비교
`Playwright`

⑥ 결과
TC 표 · 리포트 · 이전 실행과 비교
`cases-report.js`

> **핵심 원리.** 사람이 하듯 화면을 직접 눌러 보고(분석), 그때 본 결과를 테스트 코드로 고정한 뒤(생성), 새 브라우저에서 다시 해 봐서 같은 결과가 나오는지 확인합니다(실행). 기획서나 AI 모드를 쓰면 "관찰한 결과" 대신 "**원래 그래야 하는 결과**"와 비교합니다.

2. 코드 구성

파일	역할
`app/main.js`	Electron 진입점. 창을 띄우고 사용자 데이터 폴더를 정합니다.
`app/server.js`	앱의 중심. `127.0.0.1`에서만 열리는 로컬 서버로, 화면의 요청을 받아 **분석 → 생성 → 실행 → 결과 정리**를 순서대로 진행합니다. 로그인 창, 진행률, 중지도 처리합니다.
`app/public/index.html`	사용자 화면: URL · 기획서 입력, 옵션, 진행률 바, 결과.
`scripts/generate-specs.js`	**하네스(약 2,150줄).** 크롤링, 화면 분석, 클릭 관찰, 테스트 코드 생성까지 맡습니다.
`scripts/lib/codegen.js`	로케이터 · 문구 매칭 코드를 만드는 도우미.
`scripts/lib/typo.js`	규칙 기반 오타 의심 검사.
`scripts/lib/doc-specs.js`	기획서 · AI TC를 Playwright 코드로 바꿉니다.
`tests/support/site.ts`	생성된 테스트가 함께 쓰는 공통 함수: 페이지 열기 · 팝업 처리 · 오류 수집 · 접근성 · 성능 · 모바일 점검.
`app/playwright.generated.config.js`	실행 설정: 브라우저 · 동시 실행 4개 · 재시도 1회 · 트레이스 정책.
`app/llm.js` · `app/site-ai.js` · `app/doc-testcases.js`	Gemini · Claude · ChatGPT 공통 호출, AI 화면 분석, 기획서 → TC 작성과 화면 요소 연결.
`app/cases-report.js`	TC 표(`cases.html`) 생성: 대분류 · 중분류 · 소분류 · 과정 · 기대결과 · 결과 · 이유.

`app/compare-runs.js`	같은 URL의 직전 실행과 비교해 페이지 · 요소 · 동작 · 문구 변경을 정리합니다.
`app/secure-store.js`	저장된 로그인을 Electron `safeStorage`(Windows DPAPI · macOS 키체인)로 암호화합니다.
`scripts/login-session.js` · `login-check.js`	사람이 직접 로그인하는 창 / 로그인 화면인지 자동 확인.

3. 단계별로 코드에서 일어나는 일

② **페이지 찾기 (크롤러)**: 시작 URL에서 링크를 모아 세부 페이지로 들어갑니다(`CRAWL_DEPTH` 10, `MAX_PAGES` 50, 동시 3개). 서브도메인은 같은 사이트로 보지만, vercel.app 같은 호스팅 도메인은 서브도메인마다 다른 사이트로 봅니다. ID만 다른 상세 페이지와 `utm_` 같은 추적값은 하나로 합칩니다. 외부 링크는 분석하지 않고 그 사이트로 이동하는지만 확인합니다.

③ **화면 분석**: 페이지마다 새 브라우저(1440×900, 현재 PC의 언어 · 시간대)로 열어 버튼 · 링크 · 탭 · 배너 · 입력창 · 검색창 · 드롭다운 · 표 · 롤링 배너 · 툴팁 · iframe · Shadow DOM 안의 요소를 찾습니다. 그다음 실제로 눌러서 페이지 이동, 새 탭, 모달, 펼침, 탭 전환, 정렬, 콘텐츠 노출 같은 결과를 기록합니다. 클릭할 때마다 쿠키와 저장소를 처음 상태로 되돌려 오판을 막습니다. 화면 구조가 그대로인 페이지는 `analysis-cache.json`에 저장한 분석을 다시 씁니다.

④ **테스트 생성**: 화면 위에서 아래 순서로 **요소 하나 = 케이스 하나**를 만들고, 케이스 안에서 노출 · 문구 · 이미지 · 마우스 오버 · 동작을 따로 판정합니다(`checks()`). 노출이 실패하면 뒤 항목은 "확인 불가"로 건너뜁니다. 로케이터는 Playwright 권장 순서(역할+이름 → 라벨 → placeholder → 링크 → 고유 속성)를 따릅니다.

페이지마다 자동으로 추가되는 공통 점검 (실패 = 결과 실패, 경고 = 결과는 통과로 두고 경고만 표시):

점검	내용	판정
이미지 · JS 오류 · 네트워크	깨진 이미지, JS 런타임 오류, 같은 사이트의 404 · 5xx · 연결 실패	실패
iframe	임베드가 오류 페이지로 열리지 않는지	실패
콘솔 오류 · 보안	console.error, https 페이지 안의 http 리소스	경고
성능	LCP 4초 이내, 전체 로드 10초 이내	경고
모바일 (390×844)	가로로 넘치는 요소(가로 스크롤) 없음	실패
접근성 (axe-core)	WCAG 2 A/AA 위반, 상위 10개	경고
404 안내	없는 주소를 열면 404 상태나 안내 문구가 나오는지	경고

⑤ **실행**: 고른 브라우저(Chrome · WebKit · Firefox)마다 실행합니다. 동시 실행 4개, 재시도 1회이며, 재시도로 통과하면 "불안정"으로 표시합니다. 실패하면 스크린샷과 영상을 남깁니다.

⑥ **결과**: 결과 폴더 아래 `날짜-시간` 폴더에 `cases.html`, Playwright 리포트, `summary.json`, `analysis.json`, `run.log`를 저장합니다. 같은 URL의 직전 실행이 있으면 **변경 내역**(페이지 · 요소 · 동작 · 문구가 추가되거나 사라짐)을 함께 보여 줍니다.

4. 실행 모드

모드	처리	기대 결과의 기준
규칙 기반	눌러 보고 관찰 → 테스트 생성 → 실행 (AI 비용 없음)	분석 때 실제 동작 (회귀)
AI 테스트	규칙 기반 + 스크린샷 · 요소 목록을 AI에 보내 TC와 화면 이슈 추가	요소 이름 · 맥락상 당연한 동작
기획서만	AI가 기획서를 읽고 TC 표 작성 (실행 안 함)	—
URL + 기획서	기획서 TC를 화면 요소와 연결해 실행. 못 찾으면 "화면에서 못 찾음", 판단할 수 없으면 "수동 확인 필요"	기획서의 기대 결과

5. 안전장치와 로그인

- **안전 모드(기본):** 삭제 · 결제 · 제출 · 저장 · 출근처럼 데이터가 바뀌는 버튼은 누르지 않고 노출 · 문구만 확인합니다. 탈퇴 · 결제 · 송금 · 로그아웃은 "전체 동작 확인"을 켜도 누르지 않습니다.
- **로그인:** 사람이 로그인 창에서 직접 로그인하므로 **프로그램은 비밀번호를 보지 않습니다**. 저장되는 것은 쿠키 · 저장소(storageState)뿐이고, `secure-store.js`가 `safeStorage`로 암호화해 사이트별로 보관합니다. 실행할 때만 임시 파일로 풀었다가 끝나면 지웁니다. 저장 기간(`authMaxAgeDays`)이 지나면 다시 로그인하게 합니다.
- **트레이스:** 저장된 로그인으로 실행할 때는 트레이스를 끕니다. 트레이스에는 쿠키와 요청 헤더가 그대로 들어가기 때문입니다.
- **로컬 서버:** `127.0.0.1`에서만 열려 다른 PC에서는 접근할 수 없습니다.

3부

요즘 방식과 비교 · 다음 할 일

요즘 방식	QA_Automation_KC	상태
Playwright · 3개 브라우저	Chrome · WebKit · Firefox, 브라우저별 차이 표시	충족
안정적인 로케이터	역할+이름 → 라벨 → placeholder 순서	충족
상태 격리 · 불안정 관리	클릭마다 상태 복원, 재시도 후 "불안정" 표시	충족
리포트 · 증거	TC 표 · 스크린샷 · 영상 · run.log · CSV	충족
AI로 TC 작성	기획서 · 화면 기반 TC, 화면 요소와 연결	앞섬
네트워크 · JS · 콘솔 오류	페이지마다 자동 점검	충족
접근성 · 성능 · 모바일	axe-core, LCP · 로드, 390×844 넘침 점검	충족
로그인 보안	직접 로그인, 암호화 저장, 만료, 트레이스 끔	충족
변경 감지	직전 실행과 자동 비교	부분
사람이 승인한 기준(baseline)	분석 때 동작을 정답으로 자동 채택	약함
화면 비교(스크린샷 diff)	AI 화면 이슈 보고만 있음	부분
AI 실패 분류	실패 원인은 사람이 판단	없음
CI · 자동 정기 실행	CI는 설치 파일 빌드만 하고, 테스트는 앱에서 손으로 실행	없음
API 단독 테스트	화면을 거친 요청만 점검	없음

다음 할 일 (우선순위 순)

1. **헤드리스 CLI + CI 정기 실행:** `qa-kc run --url ...`처럼 앱 없이 실행하고 실패하면 종료 코드로 알리게 합니다. 그러면 GitHub Actions로 매일 밤이나 배포 후에 자동으로 돌리고 Slack · Teams로 요약을 보낼 수 있습니다.
2. **승인된 기준:** 첫 분석 결과를 사람이 검토해 승인하고, 이후 실행은 승인본과 비교합니다. 지금의 "직전 실행 비교"를 "승인본 비교"로 키우면 됩니다. 분석 때 이미 있던 버그를 정상으로 기록하는 문제가 줄어듭니다.
3. **스크린샷 비교:** 승인본 기준으로 `toHaveScreenshot()`을 쓰고, 바뀌는 영역은 가립니다.
4. **AI 실패 분류:** 실패한 단계 · 스크린샷 · 오류를 AI에 보내 실제 버그 · 사이트 변경 · 데이터 변화 · 불안정 중 무엇인지 "이유" 열에 제안하게 합니다.
5. **도구 자체 품질 측정:** 일부러 버그를 심은 샘플 사이트로 버전마다 찾아낸 버그 수와 오탐 수를 기록합니다.
6. **AI 모드 개인정보:** 로그인된 화면 캡처를 AI로 보내기 전에 동의를 받거나 민감한 영역을 가립니다. API 키도 `safeStorage`로 암호화합니다.